PRAKTIKUM PBO 5

Nama : Dzia Azizi Hamid

NIM : 251511010

Kelas : 2A-D3

Github Repo: https://github.com/Papimidz/PraktikumPBO7.git

Tujuan Pembelajaran

- Menjelaskan exception sebagai kejadian abnormal yang mengubah alur eksekusi program.
- Menggunakan try-catch untuk menangani ArrayIndexOutOfBoundsException dan NumberFormatException.
- Membuktikan bahwa posisi try-catch menentukan seberapa jauh program dapat melakukan recovery.
- Menggunakan throw untuk menolak argument yang tidak valid pada sebuah method.
- Menggunakan IllegalArgumentException beserta message yang menjelaskan penyebab kegagalan.
- Menempatkan handler sehingga program dapat melanjutkan iterasi setelah exception ditangani.
- Mengidentifikasi exception pada studi kasus sebagai RuntimeException/unchecked exception.
- Membedakan antara program yang berhenti karena exception tidak ditangani dan program yang pulih setelah exception ditangani.

Persiapan Praktikum

Struktur Project

```
praktikum7/
|-- case1-countletters/
|   `-- CountLetters.java
|-- case2-parseints/
|   `-- ParseInts.java
|-- case3-factorial/
|   |-- Factorials.java
|   `-- MathUtils.java
`-- laporan/
```

Praktikum I - Exceptions Aren't Always Errors

Kasus pertama menghitung frekuensi huruf A-Z. Setiap karakter diterjemahkan menjadi index 0..25 melalui operasi `word.charAt(i) - 'A'`. Program awal tidak memeriksa apakah karakter benar-benar huruf, sehingga spasi atau tanda baca dapat menghasilkan index di luar array.

Step 1 — Jalankan CountLetters versi awal

Tujuan: Menguji program untuk menghitung frekuensi setiap huruf dari A sampai Z.

case1-countletters > J CountLetters.java

```java
import java.util.Scanner;

public class CountLetters {
    Run | Debug
    public static void main(String[] args) {
        int[] counts = new int[26];
        Scanner scan = new Scanner(System.in);

        System.out.print("Enter a single word (letters only, please): ");
        String word = scan.nextLine();
        word = word.toUpperCase();

        for (int i = 0; i < word.length(); i++)
            counts[word.charAt(i) - 'A']++;

        System.out.println();
        for (int i = 0; i < counts.length; i++)
            if (counts[i] != 0)
                System.out.println((char)(i + 'A') + ": " + counts[i]);
    }
}
```

Pada Step 1, program CountLetters digunakan untuk menghitung frekuensi huruf dari input pengguna. Array counts memiliki 26 elemen untuk menyimpan jumlah kemunculan huruf A sampai Z. Pengujian dengan input JAVA menghasilkan A sebanyak 2 kali, J 1 kali, dan V 1 kali.

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case1-countletters> java CountLetters
Enter a single word (letters only, please): JAVA

A: 2
J: 1
V: 1
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case1-countletters>
```

Checkpoint 1: Bagaimana karakter A sampai Z diterjemahkan menjadi indeks 0 sampai 25? Mengapa array counts hanya berukuran 26?

Karakter A sampai Z diubah menjadi indeks 0 sampai 25 menggunakan operasi word.charAt(i) - 'A'. Karena terdapat 26 huruf alfabet, array counts cukup memiliki 26 elemen untuk menyimpan frekuensi setiap huruf.

Step 2 — Uji input yang mengandung spasi dan tanda baca

Tetap gunakan kode CountLetters.java dari Step 1. Pada tahap ini kita ingin melihat apa yang terjadi ketika program menerima karakter yang bukan huruf.

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case1-countletters> java CountLetters
Enter a single word (letters only, please): JAVA PBO!
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index -33 out of bounds for length 26
        at CountLetters.main(CountLetters.java:13)
```

Pada Step 2, program diuji menggunakan input JAVA PBO!. Program mengalami ArrayIndexOutOfBoundsException karena karakter spasi menghasilkan indeks array di luar rentang 0–25. Akibatnya, proses program berhenti sebelum pencetakan frekuensi huruf selesai.

Checkpoint 2: Apa yang terjadi pada alur program setelah exception muncul? Apakah loop pencetakan frekuensi huruf tetap dijalankan?

Setelah exception muncul, alur program langsung terhenti karena belum ada catch yang menangani exception tersebut. Loop untuk mencetak frekuensi huruf tidak dijalankan karena program berhenti saat memproses karakter yang tidak valid.

Step 3 - Tangkap exception dan biarkan proses berlanjut

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
        // sengaja kosong untuk eksperimen
    }
}
```

Pada Step 3, program ditambahkan try-catch untuk menangani ArrayIndexOutOfBoundsException. Karakter yang tidak valid dilewati, sehingga program tetap dapat menghitung huruf lain dan menyelesaikan proses tanpa berhenti akibat exception.

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case1-countletters> javac CountLetters.java
>> java CountLetters
Enter a single word (letters only, please): JAVA PBO!

A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Step 4 - Buat handler yang memberikan informasi berguna

Ubah body catch secara bertahap. Pertama tampilkan exception untuk debugging. Setelah itu, ubah output agar menampilkan karakter yang tidak dihitung.

```
for (int i = 0; i < word.length(); i++) {
    try {
        counts[word.charAt(i) - 'A']++;
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Not a letter: " + word.charAt(i));
    }
}
```

Pada Step 4, bagian catch diubah agar menampilkan karakter yang bukan huruf. Dengan demikian, pengguna dapat mengetahui karakter yang dilewati, sementara program tetap melanjutkan perhitungan huruf lainnya.

```
Enter a single word (letters only, please): JAVA PBO!
Not a letter:
Not a letter: !

A: 2
B: 1
J: 1
O: 1
P: 1
V: 1
```

Checkpoint 3: Apa perbedaan perilaku program sebelum dan sesudah ada catch?

Sebelum menggunakan catch, program berhenti ketika menemukan karakter yang menghasilkan indeks tidak valid. Setelah menggunakan catch, exception dapat ditangani sehingga karakter tersebut dilewati dan loop tetap memproses karakter berikutnya.

Praktikum II – Placing Exception Handlers

Step 5 - Jalankan ParseInts versi awal

```java
import java.util.Scanner;

public class ParseInts {
    Run | Debug
    public static void main(String[] args) {
        int val, sum = 0;
        Scanner scan = new Scanner(System.in);

        System.out.println("Enter a line of text");
        Scanner scanLine = new Scanner(scan.nextLine());

        while (scanLine.hasNext()) {
            val = Integer.parseInt(scanLine.next());
            sum += val;
        }

        System.out.println(
            "The sum of the integers on this line is " + sum
        );
    }
}
```

Pada Step 5, program ParseInts digunakan untuk menjumlahkan token yang berbentuk integer. Input 10 20 30 40 menghasilkan jumlah 100. Namun, input yang mengandung teks menyebabkan NumberFormatException karena token tersebut tidak dapat dikonversi menjadi integer.

```
Enter a line of text
10 20 30 40
The sum of the integers on this line is 100
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case2-parseints> java ParseInts
Enter a line of text
We have 2 dogs and 1 cat.
Exception in thread "main" java.lang.NumberFormatException: For input string: "We"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:527)
        at java.base/java.lang.Integer.parseInt(Integer.java:624)
        at ParseInts.main(ParseInts.java:12)
```

Checkpoint 4: Method apa yang melempar NumberFormatException? Token apa yang membuat method tersebut gagal?

Method Integer.parseInt() melempar NumberFormatException ketika menerima token yang bukan bilangan integer. Pada input We have 2 dogs and 1 cat., token We menyebabkan exception karena berupa teks.

Step 6 - Letakkan try-catch di luar seluruh while loop

Tujuan: Melihat apa yang terjadi ketika exception ditangani, tetapi posisi catch berada di luar loop.

```
case2-parseints > J ParseInts.java > ...
1   import java.util.Scanner;
2
3   public class ParseInts {
        Run | Debug
4       public static void main(String[] args) {
5           int val, sum = 0;
6           Scanner scan = new Scanner(System.in);
7
8           System.out.println("Enter a line of text");
9           Scanner scanLine = new Scanner(scan.nextLine());
10
11          try {
12              while (scanLine.hasNext()) {
13                  val = Integer.parseInt(scanLine.next());
14                  sum += val;
15              }
16          } catch (NumberFormatException e) {
17              // kosong untuk eksperimen
18          }
19
20          System.out.println(
21              "The sum of the integers on this line is " + sum
22          );
23      }
24  }
```

Pada Step 6, try-catch diletakkan di luar loop while. Program tidak berhenti akibat exception yang ditangani, tetapi proses perulangan berakhir ketika menemukan token yang tidak dapat dikonversi menjadi integer. Akibatnya, token setelahnya tidak ikut diproses.

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 0
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case2-parseints> java ParseInts
Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 1
```

Checkpoint 5: Mengapa token setelah exception tidak ikut diproses? Tunjukkan batas blok try pada kode.

Token setelah exception tidak diproses karena exception memindahkan alur program keluar dari blok try menuju catch. Karena seluruh loop while berada di dalam blok try, perulangan langsung berakhir ketika exception terjadi.

Step 7 - Pindahkan try-catch ke dalam loop

Tujuan: Membuat program tetap memproses token berikutnya meskipun menemukan token yang bukan integer.

```java
import java.util.Scanner;

public class ParseInts {
    Run | Debug
    public static void main(String[] args) {
        int val, sum = 0;
        Scanner scan = new Scanner(System.in);

        System.out.println("Enter a line of text");
        Scanner scanLine = new Scanner(scan.nextLine());

        while (scanLine.hasNext()) {
            try {
                val = Integer.parseInt(scanLine.next());
                sum += val;
            } catch (NumberFormatException e) {
                // token non-integer diabaikan
            }
        }

        System.out.println(
            "The sum of the integers on this line is " + sum
        );
    }
}
```

Pada Step 7, try-catch dipindahkan ke dalam loop while. Token yang bukan integer akan dilewati, sedangkan perulangan tetap berlanjut ke token berikutnya. Dengan demikian, kedua input berhasil menghasilkan jumlah 3.

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 3
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case2-parseints> java ParseInts
Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 3
```

Checkpoint 6: Mengapa posisi try-catch mengubah hasil pemrosesan token?

Jika try-catch berada di luar while, exception menyebabkan seluruh perulangan berhenti. Sebaliknya, jika berada di dalam while, exception hanya ditangani pada iterasi tersebut sehingga loop dapat melanjutkan pemrosesan token berikutnya.

Step 8 - Bandingkan dua desain handler

Step 8 meminta kita membandingkan hasil Step 6 dan Step 7. Tidak perlu membuat file baru; gunakan kode dan output yang sudah lu simpan.

Penempatan try-catch	Saat token tidak valid	Token berikutnya
Di luar while	Perulangan berakhir	Tidak diproses
Di dalam while	Exception ditangani pada iterasi tersebut	Tetap diproses

Perbedaan kedua desain terletak pada cakupan penanganan exception. Handler di luar loop menghentikan perulangan ketika terjadi exception, sedangkan handler di dalam loop memungkinkan program melanjutkan iterasi dan menjumlahkan seluruh token yang valid.

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 0
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case2-parseints> java ParseInts
Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 1
```

```
Enter a line of text
We have 2 dogs and 1 cat.
The sum of the integers on this line is 3
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case2-parseints> java ParseInts
Enter a line of text
1 fish 2 fish
The sum of the integers on this line is 3
```

Praktikum III - Throwing Exceptions

Step 9 - Jalankan baseline Factorials dan MathUtils

Tujuan: Menguji perhitungan faktorial sebelum program diberi penanganan untuk input yang tidak valid.

case3-factorial > Factorials.java > ...

```java
import java.util.Scanner;

public class Factorials {
    Run | Debug
    public static void main(String[] args) {
        String keepGoing = "y";
        Scanner scan = new Scanner(System.in);

        while (keepGoing.equals("y") || keepGoing.equals("Y")) {
            System.out.print("Enter an integer: ");
            int val = scan.nextInt();

            System.out.println("Factorial(" + val + ") = "
                    + MathUtils.factorial(val));

            System.out.print("Another factorial? (y/n) ");
            keepGoing = scan.next();
        }
    }
}
```

case3-factorial > MathUtils.java > ...

```java
public class MathUtils {
    public static int factorial(int n) {
        int fac = 1;

        for (int i = n; i > 0; i--)
            fac *= i;

        return fac;
    }
}
```

Pada Step 9, program faktorial diuji menggunakan bilangan positif, negatif, dan nilai di atas 16. Bilangan positif menghasilkan nilai yang benar, tetapi bilangan negatif menghasilkan nilai 1 yang tidak sesuai. Sementara itu, input 17 menghasilkan nilai negatif akibat overflow pada tipe data int.

```
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n) n
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case3-factorial> java Factorials
Enter an integer: -1
Factorial(-1) = 1
Another factorial? (y/n) n
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case3-factorial> java Factorials
Enter an integer: 17
Factorial(17) = -288522240
Another factorial? (y/n) n
```

Checkpoint 7: Mengapa mengembalikan nilai 1 untuk faktorial bilangan negatif berbahaya? Apa masalah jika method hanya mencetak pesan error tetapi tetap mengembalikan nilai yang dapat dianggap valid?

Mengembalikan nilai 1 untuk bilangan negatif berbahaya karena hasil tersebut dapat dianggap sebagai hasil faktorial yang benar. Jika method hanya mencetak pesan error tetapi tetap mengembalikan nilai, program pemanggil masih dapat menggunakan nilai yang salah seolah-olah valid.

Step 10 - Lempar IllegalArgumentException untuk nilai negative

Tujuan: Membuat method factorial() menolak bilangan negatif, bukan mengembalikan hasil yang keliru.

```
public class MathUtils {
    public static int factorial(int n) throws IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException(
                "factorial tidak didefinisikan untuk bilangan negatif: " + n);
        }
```

Pada Step 10, method factorial() ditambahkan throw IllegalArgumentException untuk menolak bilangan negatif. Program tidak lagi mengembalikan nilai faktorial yang keliru, tetapi menghasilkan exception dengan pesan yang menjelaskan penyebabnya. Karena belum ada catch, exception menyebabkan program berhenti.

```
PS C:\School\SEMESTER 3\PBO\PERTEMUAN 7\praktikum7\case3-factorial> javac MathUtils.java Factorials.java
>> java Factorials
Enter an integer: -1
Exception in thread "main" java.lang.IllegalArgumentException: factorial tidak didefinisikan untuk bilangan n
egatif: -1
        at MathUtils.factorial(MathUtils.java:4)
        at Factorials.main(Factorials.java:13)
```

Checkpoint 8: Siapa yang mendeteksi kondisi invalid? Siapa yang belum menangani exception? Apa akibatnya terhadap program?

Method factorial() mendeteksi input negatif dan melempar exception. Pada tahap ini, method pemanggil di Factorials.java belum menangani exception dengan catch, sehingga program berhenti ketika menerima bilangan negatif.

Step 11 - Tangkap exception di tempat yang memungkinkan recovery

Tujuan: Ketika pengguna memasukkan bilangan negatif, program menampilkan pesan kesalahan tetapi tetap bisa menerima input berikutnya.

```java
import java.util.Scanner;

public class Factorials {
    Run | Debug
    public static void main(String[] args) {
        String keepGoing = "y";
        Scanner scan = new Scanner(System.in);

        while (keepGoing.equals("y") || keepGoing.equals("Y")) {
            System.out.print("Enter an integer: ");
            int val = scan.nextInt();

            try {
                System.out.println("Factorial(" + val + ") = "
                        + MathUtils.factorial(val));
            } catch (IllegalArgumentException e) {
                System.out.println("Tidak dapat menghitung: "
                        + e.getMessage());
            }

            System.out.print("Another factorial? (y/n) ");
            keepGoing = scan.next();
        }
    }
}
```

Pada Step 11, try-catch ditambahkan di dalam loop utama untuk menangani IllegalArgumentException. Ketika input negatif dimasukkan, program menampilkan pesan kesalahan dan tetap berjalan. Pengujian dengan input 5 setelahnya menghasilkan 120, yang menunjukkan bahwa program berhasil melakukan recovery.

```
Enter an integer: -1
Tidak dapat menghitung: factorial tidak didefinisikan untuk bilangan negatif: -1
Another factorial? (y/n) y
Enter an integer: 5
Factorial(5) = 120
Another factorial? (y/n)
```

Step 12 - Tangani batas overflow int (>16)

Tujuan: Menolak input yang terlalu besar untuk perhitungan faktorial berbasis int dan memberikan pesan error yang berbeda dari input negatif.

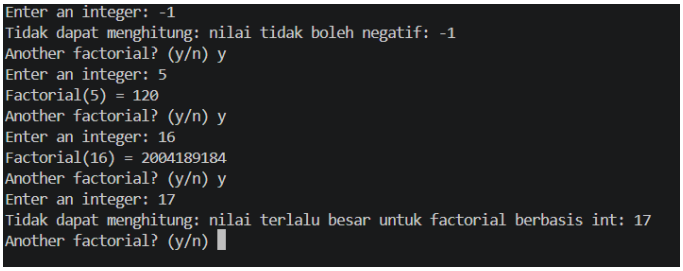
```java
public class MathUtils {
    public static int factorial(int n) throws IllegalArgumentException {
        if (n < 0) {
            throw new IllegalArgumentException(
                "nilai tidak boleh negatif: " + n);
        }

        if (n > 16) {
            throw new IllegalArgumentException(
                "nilai terlalu besar untuk factorial berbasis int: " + n);
        }

        int fac = 1;
        for (int i = n; i > 0; i--)
            fac *= i;

        return fac;
    }
}
```

Pada Step 12, method factorial() ditambahkan kondisi untuk menolak nilai di atas 16. Kedua kondisi menggunakan IllegalArgumentException, tetapi memiliki pesan berbeda agar penyebab kesalahan lebih mudah diketahui. Nilai negatif dan nilai di atas 16 ditolak, sedangkan nilai 0 sampai 16 tetap dapat diproses.

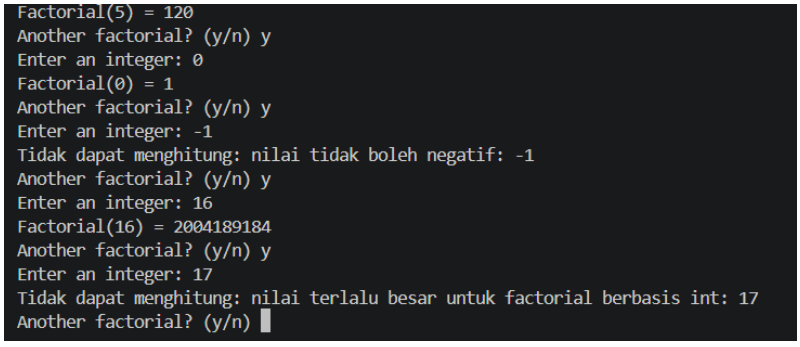


Checkpoint 9: Mengapa pesan berbeda penting ketika handler ingin menjelaskan penyebab kegagalan kepada pengguna atau developer?

Pesan yang berbeda membantu pengguna dan developer mengetahui penyebab kesalahan. Dengan begitu, mereka dapat membedakan input negatif dengan nilai yang terlalu besar tanpa harus menebak sumber masalahnya.

Step 13 - Uji batas dan recovery

Pada Step 13, program diuji menggunakan nilai 5, 0, -1, 16, dan 17. Nilai 0 sampai 16 menghasilkan faktorial yang benar, sedangkan nilai negatif dan nilai di atas 16 ditolak dengan pesan yang sesuai. Program tetap berjalan setelah exception ditangani.



ID	Skenario	Expected Result	Status	Bukti
T7-01	CountLetters dengan input JAVA	Frekuensi huruf ditampilkan tanpa exception	PASS	Screenshot output Step 1
T7-02	CountLetters dengan input JAVA PBO! sebelum catch	Muncul ArrayIndexOutOfBoundsException	PASS	Screenshot error Step 2
T7-03	CountLetters setelah ditambahkan catch	Karakter non-huruf dilewati dan program tetap berjalan	PASS	Screenshot output Step 4
T7-04	ParseInts dengan try-catch di luar while	Proses berhenti pada token tidak valid pertama	PASS	Screenshot output Step 6

T7-05	ParseInts dengan try-catch di dalam while	Semua token diproses dan jumlah integer yang valid ditampilkan	PASS	Screenshot output Step 7
T7-06	factorial(-1) sebelum ditambahkan catch	Muncul IllegalArgumentException dan program berhenti	PASS	Screenshot error Step 10
T7-07	factorial(-1) setelah ditambahkan catch	Pesan kesalahan ditampilkan dan program tetap berjalan	PASS	Screenshot output Step 11
T7-08	factorial(17) setelah validasi batas ditambahkan	Exception ditangani dengan pesan nilai terlalu besar	PASS	Screenshot output Step 12/13

Self Assessment

Pernyataan	Skor 1-5
Saya dapat membaca stack trace untuk menemukan tipe dan lokasi exception.	3
Saya dapat menulis try-catch dengan tipe exception yang tepat.	3
Saya memahami bahwa posisi handler memengaruhi scope recovery.	3
Saya dapat melempar IllegalArgumentException dengan message yang bermakna.	2
Saya dapat menempatkan catch agar program dapat melanjutkan proses setelah input invalid.	3
Saya dapat membedakan exception yang muncul dari operasi lain dan exception yang sengaja dilempar method saya sendiri.	2

Refleksi

1. Pada CountLetters, mengapa program awal berhenti saat menemukan karakter non-huruf? Apa yang berubah setelah ada catch?
   Program awal berhenti karena karakter non-huruf menghasilkan indeks array di luar batas yang tersedia. Setelah ditambahkan catch, exception bisa ditangani sehingga karakter tersebut dilewati dan program tetap melanjutkan perhitungan.
2. Apakah exception selalu berarti aplikasi harus berhenti? Gunakan satu bukti dari praktikum.
   Tidak selalu. Pada program CountLetters, setelah ditambahkan try-catch, program tetap berjalan meskipun menemukan spasi atau tanda baca karena exception sudah ditangani.
3. Mengapa try-catch di luar while pada ParseInts menghasilkan behavior berbeda dengan try-catch di dalam while?
   Karena posisi try-catch memengaruhi jalannya perulangan. Jika berada di luar while, perulangan berhenti saat menemukan token yang tidak valid. Jika berada di dalam while, token yang tidak valid dilewati dan program tetap memproses token berikutnya.
4. Dalam kasus Factorials, mengapa melempar exception lebih aman daripada mengembalikan nilai yang terlihat valid tetapi sebenarnya salah?

Karena exception memberi tahu bahwa input tidak valid sehingga hasil yang salah tidak dianggap sebagai hasil yang benar. Contohnya, faktorial bilangan negatif seharusnya ditolak, bukan menghasilkan nilai 1.

5. Apa perbedaan peran throw dan catch berdasarkan kode yang Anda jalankan?
   throw digunakan untuk melempar exception ketika terjadi kondisi yang tidak valid, sedangkan catch digunakan untuk menangani exception tersebut agar program dapat memberikan pesan kesalahan atau melanjutkan proses.
6. Apa manfaat memberikan message berbeda pada IllegalArgumentException untuk nilai negatif dan nilai >16?
   Pesan yang berbeda membantu pengguna mengetahui penyebab kesalahan. Dengan begitu, pengguna bisa membedakan apakah input negatif atau nilainya terlalu besar untuk perhitungan faktorial menggunakan int.
7. Ketiga exception utama pada modul ini termasuk unchecked exception. Apa yang ingin Anda tanyakan ketika nanti membandingkannya dengan checked exception?
   Saya ingin mengetahui apa perbedaan unchecked exception dan checked exception, terutama apakah checked exception harus ditangani dengan try-catch atau dideklarasikan menggunakan throws agar program dapat dikompilasi.
8. Lengkapi dengan bahasa Anda sendiri: "Exception handling bukan sekadar mencegah program crash, tetapi ..."
   Exception handling bukan sekadar mencegah program berhenti, tetapi juga membantu menangani kesalahan dengan tepat agar program bisa memberikan informasi yang jelas dan tetap melanjutkan proses jika memungkinkan.